cnvstrmliner

R package for parsing CNV tools' outputs

Introduction

The proliferation of Next-Generation Sequencing (NGS) tools for predicting Copy Number Aberrations (CNAs) has led to a significant data integration challenge in bioinformatics. Each application, while serving the same core purpose, generates results in a unique output format, often spread across multiple files. Furthermore, the specific metrics and fields reported—such as segment start/end positions, log-ratio values, and associated p-values—are frequently named differently or are not uniformly present across all tool outputs. This lack of standardization creates substantial overhead for researchers, who must write custom parsing scripts for every new tool, leading to complex and error-prone downstream analysis. It is exceptionally difficult to compare results across different methods, integrate them into common analysis pipelines, or use them with existing visualization and interpretation software, even when using semi-standard formats like SEG or VCF which haven't achieved universal adoption.

A dedicated package capable of reading the diverse outputs of CNA prediction tools and translating them into a single, common, standardized format is crucial for streamlining genomic data analysis. Such a package would significantly reduce the need for custom scripting, allowing researchers to focus on biological interpretation rather than data wrangling. By providing a consistent data structure, it would immediately enable method comparison, consensus calling, and seamless integration with downstream analysis and visualization tools, irrespective of the initial prediction method. This unified approach promotes reproducibility and interoperability across different studies and laboratories, ultimately accelerating the pace of discovery in cancer genomics and other fields relying on CNA analysis.

`cnvstrmliner` is a package that tries to assume the role of this middle layer that is capable of parsing multiple output formats and provide a single uniform one using standard Bioconductor objects. Furthermore, some additional functionality is added to facilitate the comparisons and QC.

Installation

The github repo is

https://github.com/bms-ips/mavrommk-cnvstrmliner.git

Since this is in the BMS (bms-ips) space we need to have a PAT to access github Store the PAT in the environment variable GITHUB_PAT

```
# potentially unload a previous copy of the package
detach("package:cnvstrmliner", unload = TRUE)

Sys.setenv("GITHUB_PAT"=YOUR_GITHUB_PAT_HERE)
devtools::install_github("caravagnalab/CNAqc")

Sys.setenv("GITHUB_PAT"=YOUR_BMS_PAT_HERE)
devtools::install_github(repo = "bms-ips/mavrommk-cnvstrmliner", force = TRUE)
```

Supporting CNV algorithms and formats

`cnvstrmliner` is able to parse outputs from the following methods

method	function	comments
SEG	parseSEG	
Sclust	parseSCLUST	Requires updateSCLUSTclone for clone specific information
ControlFreec	parseCfreec	subclonal information is buggy, thus not supported in cnvstrmlinr
FACETS	parseFACETS	ensemblCall can be used to merge multiple FACETS runs in a single object (suitable for unmatched tumor samples)
PureCN	parsePureCN	
GATK CopyNumber	parseGATK	
Battenberg	parseBattenberg	
Sequenza	parseSequenza	
CloneCNA	parseCloneCNA	
CNVkit	parseCNVkit	
Conseca	parseConseca	Aggregated CNV calls combining ControlFreec and GATK results

method	function	comments
TSV format	parseTSV	Tab-delimited files with columns: chromosome, start, end, cn1, cn2
VCF format	parseVCF	Generic VCF files with CNV information in header and INFO fields
manta	parseManta	

For examples of use see the vignette of the package and the help pages of each function.

Object description

Each cnvojb contains a GRanges slot (accessible with obj@segments) with normalized set of columns:

- total.allele.copies: the total copies found in a segment (major.allele.copies + minor.allele.copies)
- major.allele.copies: the copies of the major allele
- minor.allele.copies: the copies of the minor allele
- cnv.totalmarkers: total number of markers SNPs used to test this segment
- cnv.markers: number of heterozygous SNPs used to test this segment
- cnv.annotation: annotation of the segment (NEUT, AMP, DEL etc)
- cnv.lrr: the log2 ratio of that segment.
- cnv.cf: This is the cell fraction that has the event. Typically the purity of the tumor sample equals the highest cnv.cf.
- cnv.cff: This is the Cancer cell fraction. Events appearing in all cancer cells (CCF = 1) are considered clonal. To find the prevalence (cn.cf) of these cells in the sample divide this by the purity.
- cnv.loh: This is set to TRUE if the segment has lost its heterozygosity. Depending on the total copy number it can be a deletion, neutral LOH or even LOH in an amplified region.

Additional slots contain the information for * purity: is the predicted overall purity of the sample. * ploidy: is the ploidy of the sample * effective_ploidy: is the integer ploidy of the sample which is either provided by the tool itself or reflects the state with the majority of basepairs. * sex: the sex of the sample.

Finally if the function applyQCmetrics has been run an additional slot called cnaqcobj is provided with additional metrics about the quality of the CNA calls.

Dependencies

The package depends on a number of external packages. Most of them should be installed automatically. However, I have noticed that `svpluscnv` is not always installed.

svpluscnv has additional plotting capabilities

to install it

```
install.packages("https://cran.r-project.org/src/contrib/Archive/D3GB/D3GB_1.1.tar.gz",
                 repos = NULL)
install.packages("https://cran.r-project.org/src/contrib/Archive/taRifx/taRifx_1.0.6.2.tar.gz
                 repos = NULL)
#install.packages("remotes")
devtools::install_github("gonzolgarcia/svpluscnv")
```

Exported Functions

Parsers

- **parseBattenberg**: Parse Battenberg copy number output files integrating information from `_subclones.txt` and `_cellularity_ploidy.txt` (or `_rho_and_psi.txt`) to create a `cnvstrml` object.
- **parseCfreec**: Parse Control-FREEC output, combining information from `_info.txt`, `_CNVs`, `_ratio.txt`, and `_BAF.txt` files to construct a `cnvstrml` object with gap filling.
- **parseCNVkit**: Parse CNVkit copy number output (typically `.cns` files from the `call` command), with automatic inference of allele-specific copy numbers when missing.
- **parseConseca**: Parse Conseca copy number output (aggregated tool combining Control-Freec and GATK results), accepting data frames or files.
- **parseFACETS**: Parse FACETS copy number output from VCF or data frame format, extracting somatic CNVs, purity, and ploidy with subclone support.
- **parseGATK**: Parse GATK CopyNumber `.seg` files, inferring allele-specific copy numbers from log2 ratio values using heuristic cutoffs.
- **parseManta**: Parse Manta structural variant VCF files, extracting breakpoint ranges to create a `cnvstrml` object (no purity/ploidy information).
- **parsePureCN**: Parse PureCN output, integrating information from `<filename>.csv` (purity, ploidy) and `<filename>_loh.csv` (segments).
- **parseSCLUST**: Parse SCLUST output by combining information from `_cn_summary.txt`, `_subclonal_cn.txt`, and `_allelic_states.txt` with subclonal support.
- **parseSEG**: Parse generic SEG format files (`.seg` or `.cbs`), interpreting the last column as either copy numbers or logR values.

- **parseSequenza**: Parse Sequenza output by combining information from `_seqzinfo.txt`, `_confints_CP.txt`, and `_segments.txt`.
- **parseTSV**: Parse simple tab-delimited files with columns: `chromosome`, `start`, `end`, `cn1`, and `cn2` for custom CNV data.
- **parseCNV**: Universal parser that automatically dispatches to appropriate method-specific parser based on file format and content.
- **loadCNV**: Alias for parseCNV.
- **readCNV**: Alias for parseCNV.
- **parseVCF**: Parse VCF files containing CNV information.

Annotation

- **annotateCNV**: Annotate copy number segments with descriptive CNV type labels (HOMDEL, HETDEL, NEUT, AMP, etc.) based on allele copies and ploidy.
- **addChromosomeAnnotation**: Add chromosome, arm (p/q), centromere, and telomere annotations to segments, splitting segments that cross boundaries.
- **addLOH**: Annotate loss of heterozygosity (LOH) status based on minor allele copy number, with correct handling of sex chromosomes.
- **addCCF**: Calculate and add Cancer Cell Fraction (CCF) to segments based on cellular fraction and sample purity.

Consensus & Ensemble

- **findConsensus**: Find consensus CNV segments across multiple samples or methods, creating regions of consistent annotation with caller-specific metadata.
- **applyConsensus**: Apply pre-computed consensus segmentation to individual CNV objects for standardized breakpoint alignment.
- **ensemblCall**: Create ensemble CNV calls from multiple callers using majority voting, with ploidy adjustment and consensus value calculation.
- **mergeSegments**: Merge adjacent segments with identical copy number, annotation, and LOH status, using median/weighted averaging for values.
- **updateBattenberg_clone**: Extract and create a new `cnvstrml` object for a specific Battenberg subclone.
- **updateSCLUST_clone**: Extract and create a new `cnvstrml` object for a specific SCLUST subclone.

Comparison & Metrics

- **compareSegmentStatus**: Compare CNV status between two samples with detailed per-segment TP/FP/FN metrics at annotation, copy number, and CCF levels.

- **CNV_metrics**: Calculate pairwise Sørensen-Dice similarity metrics between CNV profiles using the CNVMetrics package, with optional state simplification.
- **CNV_entropy**: Calculate conditional entropy for CNV patterns across samples to quantify profile complexity and fragmentation.

Quality Control

- **applyQCMetrics**: Apply CNAqc quality control pipeline to assess CNV call quality using somatic mutation VAF concordance.
- **detectOverfragmentation**: Detect chromosome arms with excessive short segments using binomial tests with Bonferroni correction.
- **qcTable**: Generate comprehensive QC summary table with metrics from CNAqc analysis.

Plotting

- **plotSegments**: Visualize CNV segments across the genome with color-coded annotations, copy number tracks, and optional region highlighting for single or multiple samples.
- **plotGRanges**: Low-level plotting function for GRanges objects with flexible value and color mapping, ploidy baseline, and chromosome faceting.
- **plotCNV_freq**: Plot CNV frequency heatmap across multiple samples showing recurrent amplification and deletion patterns.

Gap Filling

- **fillGaps**: Fill genomic gaps between CNV segments with neutral-state segments to create continuous genome-wide profiles.

Export & Save

- **saveSEG** / **writeSEG**: Export CNV data to SEG format (tab-delimited) compatible with IGV and GISTIC tools.
- **writeVCF** / **saveVCF**: Export CNV data to bgzipped and tabix-indexed VCF 4.2 format.
- **writemFACETSVCF**: Write CNV data in mFACETS-compatible VCF format with specific header requirements.
- **savePhyloWGS**: Export CNV segments for PhyloWGS subclonal reconstruction analysis with NCBI chromosome format and purity adjustment.
- **saveAnnotSV**: Export CNV segments to AnnotSV-compatible BED format for structural variant annotation.

Accessors & Utilities

- **getPloidy**: Retrieve ploidy value from a `cnvstrml` object (raw fractional or rounded effective ploidy).
- **getPurity**: Retrieve tumor purity value from a `cnvstrml` object.
- **getSex**: Retrieve sex information (M/F) from a `cnvstrml` object.
- **getSupportedMethods**: Get list of all CNV calling methods supported by the package parsers.
- **summaryTable**: Generate comprehensive summary statistics at genome, chromosome, or arm levels including CNV burden and aberration frequencies.
- **matchCNVSV**: Match CNV segment boundaries with structural variant breakpoints to validate and annotate CNV calls.

External Metrics (require non-cnvstrml inputs)

- **computeCoverage**: Compute per-target coverage and genome-normalized log2 ratios from a BAM over BED/GRanges targets. Requires external BAM and BED/GRanges inputs.
- **computeGini**: Compute Gini coefficient of coverage uniformity from a coverage table (e.g., CNVkit `.cnr`). Requires a data frame with a `depth` column.
- **computeMAPD**: Compute Median Absolute Pairwise Difference (MAPD) from log2 ratio data (e.g., CNVkit `.cnr`). Requires columns `chromosome` and `log2`.

Utility scripts

postCNV.R

Location: `inst/extdata/postCNV.R`

Purpose: Comprehensive post-processing script for CNV calls from various tools. Acts as a workhorse for standardizing CNV outputs and computing quality control metrics.

Key Features: - **Format Conversion:** Converts CNV calls from any supported method to standardized VCF 4.2 format (bgzipped and tabix-indexed) - **Quality Control:** Computes CNAqc metrics when somatic mutations VCF is provided - **CNV Entropy:** Calculates genomic fragmentation metrics using covered regions from BED files - **Coverage Metrics:** Computes Gini coefficient and MAPD (Median Absolute Pairwise Difference) from BAM files - **Multi-format Output:** Saves results as VCF, RDS (cnvstrml object), and QC statistics tables

Usage:

```
./postCNV.R -i <input_file> -m <method> -o <output.vcf.gz> [options]
```

Options: - `-i, --input`: Path to the input CNV file (required) - `-m, --method`: CNV calling method - FACETS|Sequenza|SCLUST|Battenberg|FREEC|CNVkit|PureCN|GATK|Conseca|CloneCNA|TSV|VCF|SEG (required) - `-o, --output`: Path to the output VCF file (required) - `-s, --sample`: Sample name (optional, inferred from input if not provided) - `-v, --somatic_mutations`: Somatic mutations VCF for CNAqc quality control (optional) - `-b, --bam`: BAM file for Gini and MAPD coverage metrics calculation (optional) - `-d, --bed`: BED file with covered regions for CNV entropy and coverage metrics (optional) - `-h, --help`: Show help message

Output Files: 1. **`<sample>-<method>.vcf.gz`** - Standardized CNV calls in VCF 4.2 format (bgzipped) 2. **`<sample>-<method>.vcf.gz.tbi`** - Tabix index for the VCF file 3. **`<sample>-<method>.rds`** - R data file containing the cnvstrml object 4. **`<sample>-<method>_qc_stats.tsv`** - QC metrics table (if somatic mutations provided) 5. **`<sample>-<method>_entropy.tsv`** - CNV entropy metrics (if BED file provided) 6. **`<sample>-<method>_coverage_metrics.tsv`** - Gini and MAPD metrics (if BAM file provided)

Example:

```
# Basic conversion
./postCNV.R -i sample1_facets.vcf.gz -m FACETS -o sample1_processed.vcf.gz

# With full QC metrics
./postCNV.R -i sample1.cns -m CNVkit -o sample1_cnv.vcf.gz \
  -v sample1_somatic.vcf.gz \
  -b sample1.bam \
  -d targets.bed \
  -s SAMPLE001
```

---

annotateGenes.R

Location: `inst/extdata/annotateGenes.R`

Purpose: Annotate CNV segments with overlapping gene information using Ensembl annotations from AnnotationHub.

Key Features: - **Gene-Level Annotation:** Maps CNV segments to individual genes with copy number status. For genes overlapping multiple segments the weighted copy number is provided according to: -

$$\text{Gene CN} = \frac{\sum(\text{Segment CN}_i \times \text{Overlap Width}_i)}{\text{Total Overlap Width}}$$

- **Segment-Level Annotation:** Provides segment-wise CNV information with genotype representation
- **Ensembl Integration:** Uses AnnotationHub with Ensembl database release 109 (hg38/GRCh38)
- **LOH Detection:** Annotates loss of heterozygosity including copy-neutral LOH events
- **Standardized Output:** Generates tab-delimited files compatible with downstream analysis tools

Usage:

```
./annotateGenes.R -i <input.vcf.gz> -c <cache_dir> -o <output_dir> [options]
```

Options: - `-i, --input`: Path to CNV calls in VCF format from cnvstrmliner (required) - `-c, --cache`: Path to AnnotationHub cache directory (required, or set via `ANNOTATION_HUB_CACHE` environment variable) - `-o, --output_directory`: Output directory for annotation files (default: current directory) - `-s, --sample`: Sample name (optional, extracted from VCF if not provided) - `-h, --help`: Show help message

Output Files: 1. `<sample>-<method>.gene_level.tsv` - Gene-level CNV annotations - Columns: `chr`, `start`, `end`, `gene_name`, `Status`, `Ratio`, `Absolute.Copy.Number`, `Sample.Tumor.Purity`, `LOH` - Only includes genes with non-neutral CNV status - Status categories: Deep deletion, Shallow deletion, Amplification, Neutral - LOH categories: "copy-neutral LOH", "LOH: loss of heterozygosity", or empty

2. `<sample>-<method>.segment_level.tsv` - Segment-level CNV annotations
    - Columns: `chr`, `start`, `end`, `Status`, `Ratio`, `Absolute.Copy.Number`, `Sample.Tumor.Purity`, `LOH`, `Genotype`
    - Genotype format: allelic representation (e.g., "AAB" = 2 copies of A allele, 1 copy of B allele)
    - Excludes neutral segments for focused analysis

Prerequisites: - AnnotationHub cache must be pre-downloaded with Ensembl release 109 for Homo sapiens - Input VCF must be generated by cnvstrmliner (contains required metadata)

Example:

```bash
# Set cache directory and annotate CNVs
export ANNOTATION_HUB_CACHE=/path/to/annotation_cache
./annotateGenes.R -i sample1_cnv.vcf.gz -c $ANNOTATION_HUB_CACHE -o ./annotations/

# With explicit sample name
./annotateGenes.R -i sample1_cnv.vcf.gz -c /data/annotation_hub -o ./output/ -s SAMPLE001
```

Output Example (gene_level.tsv):

chr	start	end	gene_name	Status	Ratio	Absolute.Copy.Number
1	100000	200000	GENE1	Amplification	0.58	5
3	500000	600000	GENE2	Shallow deletion	-0.32	1
7	1000000	1200000	GENE3	Deep deletion	-2.15	0